

Blatt 8

Web-Security und Intrusion Prevention

Luis Staudt

Vorbemerkung

Ein Teil dieser Aufgabe setzt externe Infrastruktur voraus, die lokal nicht reproduzierbar ist: eine bwCloud-VM mit öffentlicher IP (Login über HFU-Account), einen öffentlichen DNS-Namen über duckdns.org und ein öffentlich validiertes Let's-Encrypt-Zertifikat, dessen ACME-Challenge von aussen erreichbar sein muss. Dieser Teil wurde nicht ausgeführt.

Der zentrale Lerninhalt, der MITM-Angriff per ARP-Spoofing und der Schutz durch HTTPS, wurde dagegen in einem lokalen, self-contained Docker-Lab durchgeführt (Kapitel „Lokale Reproduktion“), mit einem lokalen Login-Server anstelle der bwCloud-Seite. HSTS und die Let's-Encrypt-Validierung werden zusätzlich erklärt. Das Lab liegt reproduzierbar unter lokal-lab/ (run-mitm.sh).

Aufgabe 1: Die Anwendungen

Die lokale Angriffsumgebung besteht aus drei Docker-Containern:

- **browser** – das Opfer. Ein Firefox, der über VNC beziehungsweise eine Web-Oberfläche (<http://10.0.0.10:5800/>) bedient wird und sich über den Router mit dem Internet verbindet.
- **router** – stellt die Internet-Verbindung her (NAT/MASQUERADE, `net.ipv4.ip_forward=1`). Default-Gateway ist 90.0.255.254.
- **attacker** – der Angreifer mit den Werkzeugen `dsniff`/`arpspoof`, `tcpdump`, `scapy` und `bettercap`. Führt den MITM-Angriff durch.

Der Webserver (Immich) läuft getrennt davon auf der bwCloud-VM.

Aufgabe 2: VM und Webanwendung (bwCloud)

Die folgenden Schritte wurden auf der externen bwCloud-Infrastruktur ausgeführt, nicht im lokalen Lab:

- SSH-Schlüssel erzeugen (`ssh-keygen`, mit Passwort gesichert); der private Schlüssel bleibt geheim, `id_rsa.pub` wird in die VM importiert. Der Passwort-Login ist deaktiviert, da Passwörter leicht angreifbar sind.
- Debian-13-VM (m1.medium, public-v4) in bwCloud erstellen, per SSH verbinden (`ssh debian@<IP>`), Updates einspielen, Docker installieren und den Benutzer `debian` zur `docker`-Gruppe hinzufügen.
- Domainnamen über duckdns.org registrieren und die statische VM-IP eintragen.
- Immich per `docker-compose` installieren, Port vorübergehend auf 80 stellen und einen Testbenutzer anlegen, mit Pseudonym und Wegwerf-Passwort, da die Verbindung zunächst unverschlüsselt und mitlesbar ist.

Aufgabe 3: MITM-Angriff per ARP-Spoofing

ARP kennt keine Authentifizierung. Der Angreifer sendet gefälschte ARP-Antworten und bringt so Opfer und Gateway dazu, seine MAC-Adresse für die jeweils andere IP einzutragen. Dadurch läuft der gesamte Verkehr über den Angreifer (Man-in-the-Middle).

Im attacker-Container wird zunächst das Default-Gateway gesetzt, da andernfalls die Internet-Verbindung des Opfers abbricht:

```
1 docker exec -it attacker bash
2 route add default gw 90.0.255.254
```

Das ARP-Spoofing erfolgt in beide Richtungen: dem Opfer 90.0.0.1 gegenüber als Gateway und dem Gateway gegenüber als Opfer:

```
1 # gegenueber dem Opfer als Gateway auftreten
2 arpspoof -i eth0 -t 90.0.0.1 90.0.255.254
3 # gegenueber dem Gateway als Opfer auftreten
4 arpspoof -i eth0 -t 90.0.255.254 90.0.0.1
```

Da Immich zunächst über HTTP läuft, stehen die Login-Daten im Klartext im Paket. tcpdump zeigt die Paketinhalte an:

```
1 tcpdump -i eth0 -A -n 'tcp port 80'
```

Loggt sich das Opfer nun auf der Webseite ein, erscheint das im Klartext übertragene Passwort im Mitschnitt (POST-Body des Login-Requests). Danach werden arpspoof und tcpdump beendet.

Aufgabe 4: Schutz mit Verschlüsselung und Zertifikaten

Immich wird auf Port 2283 zurückgestellt und ist nur noch intern erreichbar. Davor kommt der *NginX Proxy Manager* (NPM), der über Port 80/443 nach aussen spricht und per Let's Encrypt ein Zertifikat für die duckdns-Domain anfordert. Zusätzlich werden *Force SSL* und *HSTS* aktiviert. Port 81 (NPM-Admin) muss in der bwCloud-Firewall freigegeben werden.

Bedeutung von HSTS

HSTS (HTTP Strict Transport Security) ist ein Response-Header (Strict-Transport-Security: max-age=...), mit dem der Server dem Browser mitteilt, die Seite für die angegebene Dauer ausschliesslich über HTTPS aufzurufen. Daraus ergeben sich folgende Konsequenzen:

- Der Browser ersetzt `http://`-Aufrufe der Domain intern durch `https://`, noch bevor eine Anfrage das Netz verlässt.
- Ein erzwungenes Downgrade auf HTTP (SSL-Stripping) wird verhindert.
- Bei Zertifikatsfehlern ist kein „Trotzdem fortfahren“ mehr möglich.

HSTS schliesst damit das Zeitfenster der ersten, noch unverschlüsselten Anfrage, mit preload sogar vollständig.

Wie Let's Encrypt den Besitz des Servers prüft

Let's Encrypt nutzt das ACME-Protokoll mit einer Domain-Validation-Challenge. Der Antragsteller muss nachweisen, dass er die Domain kontrolliert:

- **HTTP-01:** Let's Encrypt gibt ein Token vor; der Server muss es unter `http://<domain>/.well-known/acme-challenge/<token>` bereitstellen. Let's Encrypt ruft diese URL über den öffentlichen DNS-Namen ab. Nur wer den Server hinter der Domain kontrolliert, kann die Datei dort ablegen.
- **DNS-01:** Alternativ wird ein vorgegebener Wert als TXT-Record (`_acme-challenge.<domain>`) im DNS hinterlegt.

Ein Angreifer ohne Kontrolle über Domain beziehungsweise Server kann die Challenge nicht erfüllen und erhält daher kein gültiges Zertifikat. NPM übernimmt diesen Ablauf automatisch.

MITM nach der Umstellung

Wird der Angriff bei aktivem HTTPS wiederholt, sieht der Angreifer nur noch TLS-verschlüsselten Verkehr; das Passwort ist nicht mehr auslesbar. Zwar könnte er die Pakete weiterhin umleiten, ohne den privaten Schlüssel des Servers aber den Inhalt nicht entschlüsseln. Der Versuch, ein eigenes Zertifikat unterzuschieben, scheitert an der Zertifikatsprüfung des Browsers und zusätzlich an HSTS.

Lokale Reproduktion (Docker)

Da die bwCloud-Seite extern ist, wurde der Angriff in einem isolierten Docker-Netz (192.168.80.0/24) nachgestellt. Es kamen drei Container zum Einsatz: server (.10, Login-Webseite), victim (.20) und attacker (.66).

Aufgabe 3 lokal: ARP-Spoofing + Klartext-Passwort

Der Angreifer aktiviert IP-Forwarding und fälscht per arpspoof die ARP-Zuordnung in beide Richtungen:

```
1 sysctl -w net.ipv4.ip_forward=1
2 arpspoof -i eth0 -t 192.168.80.20 192.168.80.10 # dem Opfer: ich bin der Server
3 arpspoof -i eth0 -t 192.168.80.10 192.168.80.20 # dem Server: ich bin das Opfer
```

Danach zeigt die ARP-Tabelle des Opfers für die Server-IP die MAC des Angreifers, sodass der Verkehr nun über den Angreifer läuft:

```
1 # victim /proc/net/arp
2 192.168.80.10 ... 56:24:2a:f5:f7:52 * eth0 # = MAC des Angreifers
```

Loggt sich das Opfer nun per HTTP ein, liest der Angreifer das Passwort im Klartext aus dem Mitschnitt (images/mitm_http.pcap):

```
1 tcpdump -nr mitm.pcap -A | grep password=
2 -> user=alice&password=SuperGeheim123
```

Aufgabe 4 lokal: mit HTTPS ist das Passwort geschützt

Derselbe Angriff richtet sich gegen einen TLS-Server (Port 443, selbstsigniertes Zertifikat). Das Opfer loggt sich jetzt über HTTPS ein. Der Angreifer sieht zwar die Pakete, aber nur TLS-verschlüsselt:

```
1 # Vorkommen des Passworts im jeweiligen Mitschnitt:
2 grep -c SuperGeheim (HTTP-Mitschnitt mitm.pcap) -> 4 (lesbar)
3 grep -c SuperGeheim (HTTPS-Mitschnitt mitm_tls.pcap) -> 0 (nicht lesbar)
```

Ohne Verschlüsselung ist das Passwort trivial mitlesbar, mit HTTPS nicht. Dies deckt sich mit dem erwarteten Verhalten auf der bwCloud. Die pcap-Dateien liegen unter images/ bei.

Zertifikatsausstellung per ACME (lokal mit Pebble)

Ein echtes Let's-Encrypt-Zertifikat erfordert eine öffentliche Domain, deren ACME-Challenge von aussen erreichbar ist, was lokal nicht möglich ist. Stattdessen wurde *Pebble*, der offizielle Test-ACME-Server von Let's Encrypt, verwendet. Pebble spricht dasselbe ACME-Protokoll (Order, HTTP-01, Ausstellung) wie die produktive CA; lediglich die ausstellende CA ist eine Test-CA. So lässt sich der vollständige Ablauf real durchführen.

Die Ausstellung erfolgt über `acme.sh` im Standalone-Modus. Pebble validiert die HTTP-01-Challenge, indem es `http://<domain>:5002/.well-known/acme-challenge/<token>` abruft, also genau die Datei, die der ACME-Client während der Ausstellung bereitstellt:

```
1 acme.sh --issue --standalone --httpport 5002 -d immich.websec.lab \
2   --server https://pebble:14000/dir
3 -> Order -> HTTP-01 challenge -> finalize -> Cert success.
```

Das ausgestellte Zertifikat zeigt im Auszug (`images/acme_cert.txt`):

```
1 issuer=CN=Pebble Intermediate CA 1a139c
2 X509v3 Subject Alternative Name: DNS:immich.websec.lab
3 notBefore=Oct  6 11:27:58 2026   notAfter=Jan  4 11:27:57 2027
4 # Kette: Leaf <- Pebble Intermediate CA <- Pebble Root CA
```

Für die Vertrauensprüfung wird der Webserver (NginX) mit dem ausgestellten Zertifikat betrieben. Ein Client, der die zugehörige Root-CA kennt, akzeptiert die Verbindung ohne Warnung; ohne sie schlägt die Prüfung fehl:

```
1 curl https://immich.websec.lab/ -> ssl_verify_result=20 (unbekannte CA)
2 curl --cacert root https://immich.websec.lab/ -> ssl_verify_result=0, HTTP 200
```

Das ist exakt der Mechanismus, den auch Let's Encrypt nutzt: Ein über ACME ausgestelltes, von einer vertrauenswürdigen CA signiertes Zertifikat macht die Verbindung für den Browser gültig und schützt, wie oben gezeigt, gegen das Mitlesen durch den MITM-Angreifer. Das Setup liegt reproduzierbar unter `pebble-lab/` (`run-acme.sh`).